

Simulasi Prediksi Stok

Panduan Simulasi Prediksi Stok — End-to-End

Dokumen ini memandu simulasi lengkap sistem dari awal hingga akhir tanpa perlu perangkat WhatsApp sungguhan. Semua interaksi WhatsApp dilakukan via HTTP API (`curl`).

Daftar Isi

1. Prasyarat & Setup Awal	
2. Menjalankan Sistem	
3. Skenario Simulasi	
◦ 3a. Catat Penjualan (terjual)	
◦ 3b. Cek Status Stok	
◦ 3c. Konfirmasi Stok Akhir Hari (cek stok)	
◦ 3d. Restok	
◦ 3e. Prediksi & Forecast	
◦ 3f. Dashboard	
4. Skenario Lengkap (Copy-Paste)	
5. Memahami Output	

1. Prasyarat & Setup Awal

Prasyarat

Tools	Minimal
Python	3.14 (via <code>uv</code>)
Node.js	18+

Tools	Minimal
uv	lihat docs.astral.sh/uv

Setup Cepat

```
# 1. Clone & masuk direktori (skip jika sudah)
cd /home/amri/Projects/prediksi_stok
```

```
# 2. Install Python deps
uv sync
```

```
# 3. Install Node deps (WhatsApp bot)
cd whatsapp-bot && npm install && cd ..
```

```
# 4. Generate synthetic data + train models
uv run python -c "
from app.seeder import seed_synthetic_data
from app.predictor import train_all_products
```

```
DB_PATH = 'data/prediksi.db'
PRODUCTS_FILE = 'products.json'
import os; os.makedirs('data', exist_ok=True)
```

```
seed_synthetic_data(DB_PATH, PRODUCTS_FILE, days=90)
train_all_products(DB_PATH, PRODUCTS_FILE)
print('Siap – data sintetis 90 hari + 7 model Prophet.')
"
```

Catatan: Jika database sudah berisi data nyata, backup dulu sebelum menjalankan ulang seeder.

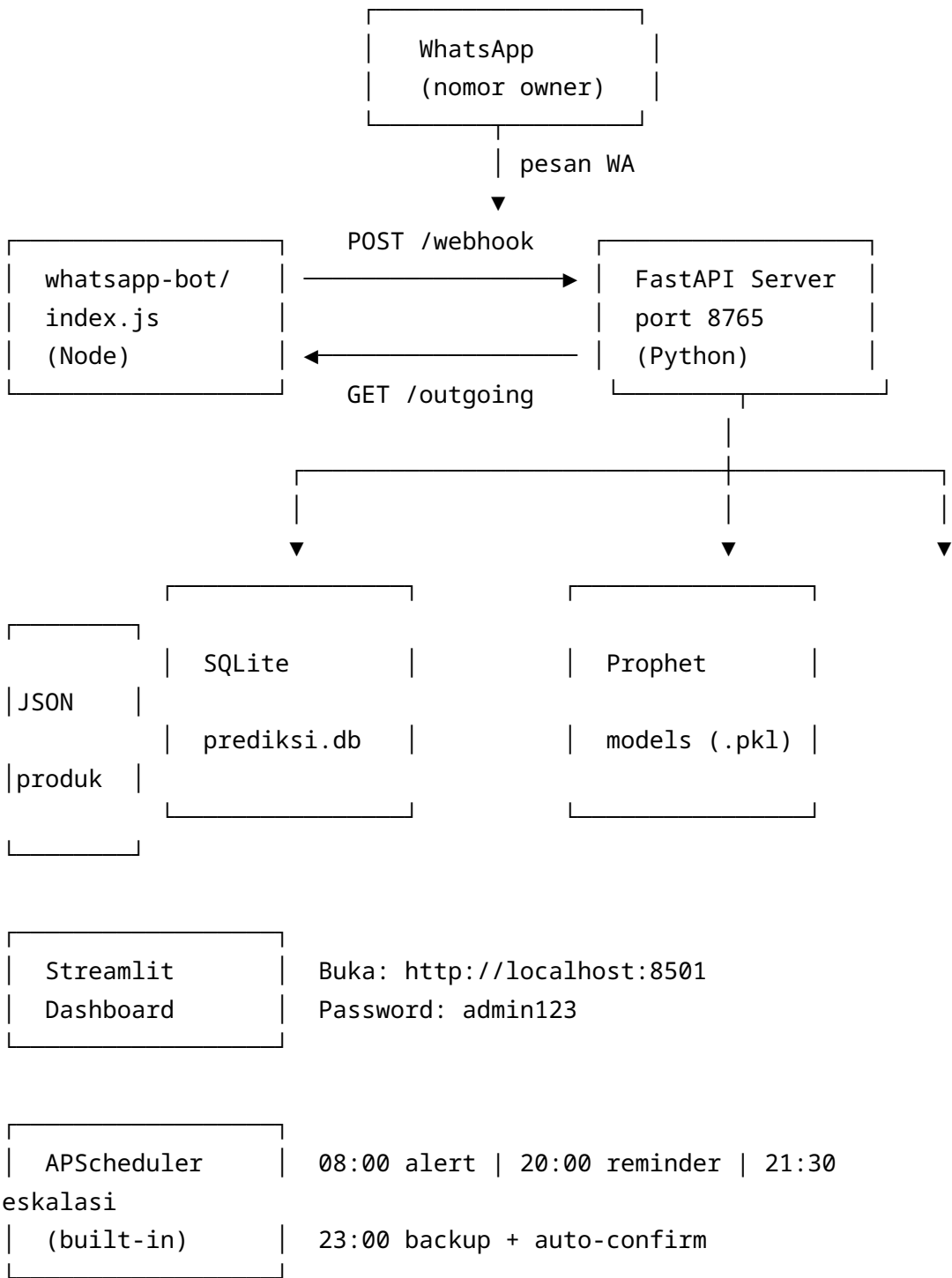
Reset ke Keadaan Awal (Opsional)

```
# Hapus database & model, lalu setup ulang
rm -f data/prediksi.db data/models/*.pkl
uv run python -c "
from app.seeder import seed_synthetic_data
from app.predictor import train_all_products
import os; os.makedirs('data/models', exist_ok=True)
seed_synthetic_data('data/prediksi.db', 'products.json', days=90)
train_all_products('data/prediksi.db', 'products.json')
```

```
print('Reset + seeded 90 days synthetic data')
"
```

2. Menjalankan Sistem

Arsitektur Sistem



Terminal

Terminal 1 — FastAPI Server (Port 8765)

```
cd /home/amri/Projects/prediksi_stok
uv run uvicorn app.server:app --host 0.0.0.0 --port 8765 --reload
```

Cek:

```
curl -s http://localhost:8765/health
# → {"status":"ok"}
```

Terminal 2 — WhatsApp Bot (Opsional, untuk QR)

```
cd /home/amri/Projects/prediksi_stok/whatsapp-bot
OWNER_NUMBER=+6281279454414 node index.js
```

Bot ini akan: 1. Generate QR code (data/qr.png) 2. Polling /outgoing setiap 3 detik untuk pesan keluar 3. POST pesan masuk ke /webhook

Tanpa WhatsApp sungguhan, simulasi tetap berjalan via curl ke /webhook.

Terminal 3 — Dashboard Streamlit

```
cd /home/amri/Projects/prediksi_stok
uv run streamlit run dashboard.py --server.port 8501
```

Buka browser: <http://localhost:8501> Password: admin123

3. Skenario Simulasi

Semua endpoint ada di <http://localhost:8765>.

3a. Catat Penjualan

Format: `terjual <produk> <jumlah>[, <produk> <jumlah>...]`

Produk valid: gula, minyak, tepung, beras, aqua, roti hitam manis, garam

```
# Penjualan tunggal
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
```

```

-d '{"from_number": "+6281279454414", "body": "terjual gula 5"}'

# Multi produk
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number": "+6281279454414", "body": "terjual gula 3,
      minyak 20, beras 50"}'

# Auto-correct: spasi terlekat
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number": "+6281279454414", "body": "terjual gula5
      minyak20"}'

# Produk tidak dikenal → error
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number": "+6281279454414", "body": "terjual indomie
      10"}'

# Jumlah mencurigakan (>10x estimasi harian)
# estimasi gula = 30/7 ≈ 4.3, maka 50 > 43 → flagged
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number": "+6281279454414", "body": "terjual gula
      50"}'

```

Response contoh (sukses):

```

{
  "status": "ok",
  "response":
    "OK. Gula +3 sak (total hari ini: 8), Minyak +20 dus
    (total hari ini: 20), Beras +50 kg (total hari ini: 50)"
}

```

3b. Cek Status Stok

```

# Status semua produk
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number": "+6281279454414", "body": "cek stok"}'

```

Response contoh:

Gula: 27 sak | habis: 2026-06-12 ↓ | BL/low
 Minyak: 980 dus | habis: 2026-06-19 ↓ | BL/low

✓ Tepung: 300 dus | habis: >30 hari → | BL/med

...

Legend: | Ikon | Arti | |——|——| | (spasi) | confidence low | | ? | confidence medium | | ?? | tidak ada prediksi | | ↑ ↓ → | tren naik/turun/stabil | | B / BL / M | fase bootstrap / blend / mature |

3c. Konfirmasi Stok Akhir Hari — Tiga Tahap

Sistem punya 3 tahap konfirmasi stok akhir hari:

20:00 → Reminder dengan expected stock

21:30 → Eskalasi (produk yang belum dikonfirmasi)

23:00 → Auto-confirm untuk semua yang tersisa

Tahap 1 — Reminder (20:00)

Bot otomatis kirim pesan ke owner dengan **expected stock per produk**:

Waktunya cek stok!

Expected stok hari ini:

Gula: 25 sak

Minyak: 980 dus

Tepung: 290 dus

Beras: 2960 kg

Aqua: 300 dus

Roti hitam manis: 34 dus

Garam: 495 pak

Balas: "ok" jika semua sesuai

Atau: gula 25, minyak 950, ...

Respon A — Owner setuju semua:

```
curl -X POST http://localhost:8765/webhook \  
-H "Content-Type: application/json" \  
-d '{"from_number":"+6281279454414","body":"ok"}'
```

Response:

Stok hari ini dikonfirmasi:

✓ Gula: 25 sak

✓ Minyak: 980 dus

✓ Tepung: 290 dus

...

Ada selisih? Kirim "cek stok [produk] [jumlah]" untuk koreksi.

Respon B — Owner konfirmasi manual (seperti sebelumnya):

```
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number": "+6281279454414", "body": "cek stok gula
    25, minyak 950"}'
```

Response:

Stok diperbarui. Gula: 25 sak (sesuai); Minyak: 950 dus (sesuai)

Tahap 2 — Eskalasi (21:30)

Jika ada produk yang belum dikonfirmasi, bot kirim:

Stok BELUM dikonfirmasi untuk: Tepung, Beras, Aqua, ...
Kirim "ok" jika semua sesuai,
atau kirim: cek stok [produk] [jumlah]

Produk yang sudah dikonfirmasi di tahap 1 tidak akan disebut lagi.

Tahap 3 — Auto-confirm (23:00)

Jika masih ada produk yang belum dikonfirmasi, bot auto-confirm dengan nilai expected:

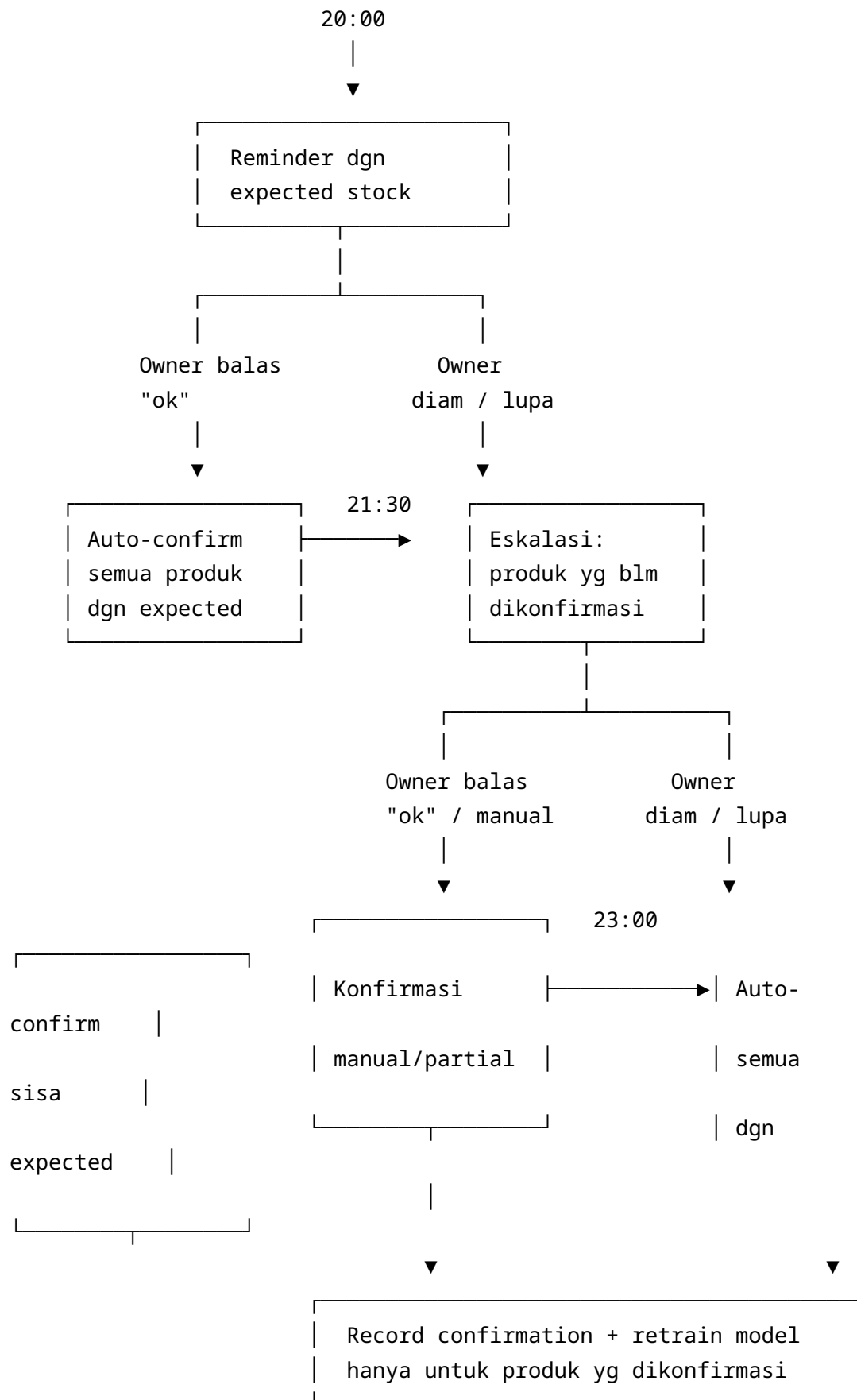
Stok otomatis dikonfirmasi:
✓ Tepung: 290 dus (expected)
✓ Beras: 2960 kg (expected)
...

Ada selisih? Kirim "cek stok [produk] [jumlah]" besok pagi.

Logika rekonsiliasi (berlaku untuk konfirmasi manual & auto): -
sesuai — selisih $\leq 10\%$ atau ≤ 1 unit (max keduanya) - susut — stok
aktual $<$ expected (hilang/rusak/curi) - restok — stok aktual $>$ expected
(pemasok datang tanpa dicatat)

Setiap konfirmasi memicu **retraining model Prophet** hanya untuk
produk yang dikonfirmasi (bukan semua 7 produk).

Flowchart Konfirmasi Akhir Hari



3d. Restok

Format: restock <produk> <jumlah>

```
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number": "+6281279454414", "body": "restock aqua 200"}'
```

Response:

```
{"status": "ok", "response": "OK. Aqua di-restok 200 dus. Stok sekarang: 480 dus."}
```

3e. Prediksi & Forecast

Semua produk:

```
curl -s http://localhost:8765/predict | jq
```

Satu produk:

```
curl -s http://localhost:8765/predict/Gula | jq
```

Response:

```
{
  "product": "Gula",
  "depletion_days": 7,
  "depletion_date": "2026-06-12",
  "phase": "blend",
  "trend": "down",
  "confidence": "low",
  "fallback_active": false
}
```

Field penting: | Field | Arti | |——|——| | depletion_days |
Perkiraan hari sampai stok habis (null jika >30) | | depletion_date |
Tanggal perkiraan habis | | phase | bootstrap / blend / mature | |
confidence | low (bootstrap/fallback), medium (blend), high (mature) | |
trend | up / down / stable | | fallback_active | True jika model Prophet
gagal, pakai estimasi sederhana |

3f. Dashboard

Buka <http://localhost:8501>, login dengan admin123.

Fitur dashboard: 1. **Tabel stok** — semua produk dengan stok, prediksi habis, tren, fase, status 2. **Grafik** — pilih produk, lihat bar chart penjualan 30 hari + line chart prediksi 30 hari 3. **Status WhatsApp** — koneksi, QR re-pairing 4. **CRUD Produk** — tambah/edit/hapus produk via UI (termasuk regenerasi data sintetis)

Dashboard auto-refresh setiap 10 detik.

4. Skenario Lengkap (Copy-Paste)

Simulasi satu hari penuh — jalankan berurutan:

— 1. Cek kesehatan server —

```
curl -s http://localhost:8765/health
```

— 2. Pagi: catat penjualan —

```
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number":"+6281279454414","body":"terjual gula 2, minyak 15"}'
```

```
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number":"+6281279454414","body":"terjual beras 40, tepung 5"}'
```

— 3. Siang: cek stok —

```
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number":"+6281279454414","body":"cek stok"}'
```

— 4. Sore: lebih banyak penjualan —

```
curl -s -X POST http://localhost:8765/webhook \
  -H "Content-Type: application/json" \
  -d '{"from_number":"+6281279454414","body":"terjual gula 1, aqua 10, garam 5"}'
```

— 5. Cek prediksi —

```
curl -s http://localhost:8765/predict/Gula | jq .
```

```
curl -s http://localhost:8765/predict/Beras | jq .
```

— 6. Restok (jika hampir habis) —

```
curl -s -X POST http://localhost:8765/webhook \
```

```

-H "Content-Type: application/json" \
-d '{"from_number":"+6281279454414","body":"restock roti hitam manis 30"}'

# — 7. Akhir hari: konfirmasi stok —
# Opsi A: "ok" jika semua sesuai
curl -s -X POST http://localhost:8765/webhook \
-H "Content-Type: application/json" \
-d '{"from_number":"+6281279454414","body":"ok"}'

# Opsi B: manual untuk produk tertentu
curl -s -X POST http://localhost:8765/webhook \
-H "Content-Type: application/json" \
-d '{"from_number":"+6281279454414","body":"cek stok gula 27, minyak 965, tepung 295, beras 2960, aqua 290, roti hitam manis 30, garam 495"}'

# — 8. Cek prediksi setelah retraining —
curl -s http://localhost:8765/predict | jq 'to_entries[] | {produk: .key, habis: .value.depletion_date, fase: .value.phase, confidence: .value.confidence}'

```

5. Memahami Output

Fase Prediksi

Fase	Kondisi	Data yang Dipakai	Confidence
bootstrap	Baru mulai, 0 hari real	Synthetic 100%	low
blend	1–59 hari real	Synthetic + real (90 hari rolling)	medium
mature	≥60 hari real	Real only	high

Rekonsiliasi Stok

Rumus: $\text{expected} = \text{last_confirmed} - \text{total_penjualan_sejak_konfirmasi}$

Perbandingan: $\text{delta} = \text{actual} - \text{expected}$

Kondisi	Deteksi
$ \text{delta} \leq \max(10\% \times \text{expected}, 1 \text{ unit})$	Match (sesuai)
$\text{delta} < -\text{threshold}$	Susut (shrinkage)
$\text{delta} > \text{threshold}$	Restok tak tercatat

Scheduler Otomatis

Waktu	Aksi
08:00	Cek semua produk → jika depletion $\leq$ lead time, kirim alert
20:00	Kirim reminder EOD + expected stock per produk
21:30	Eskalasi untuk produk yang belum dikonfirmasi
23:00	Backup database + auto-confirm stok (expected) + retrain model
(setelah konfirmasi)	Retrain Prophet hanya untuk produk yang dikonfirmasi

Troubleshooting

Masalah	Solusi
uv sync gagal	Pastikan Python 3.14 terinstall (uv python list)
Port 8765 已 dipakai	Set FASTAPI_PORT=8766 di .env Buka dashboard, dia auto-train. Atau: uv run python -c "from app.predictor import train_all_products; train_all_products('data/prediksi.db', 'products.json')"
Dashboard error model	
Ingin data fresh	Reset database (lihat section 1)
Webhook 404	

Masalah	Solusi
Pesan tidak terkirim	Pastikan FastAPI running: curl -s http://localhost:8765/health Bot WhatsApp tidak jalan — simulasi via curl tetap bisa